AWS ARCHITECTURE REFERENCE

AgentCore Memory
Architecture Guide

Strands Framework · EU Banking · GDPR Compliance

A comprehensive, prescriptive reference covering all memory types,
multi-agent patterns, token optimisation, framework comparisons,
EU banking compliance, security policy, Terraform IaC and project journey.

Short-Term Memory | Long-Term Memory | Episodic Memory | Multi-Agent | GDPR / DORA | Token Optimisation | Terraform IaC

VERSION	DATE	STATUS	CLASSIFICATION
2.0	April 2026	PRODUCTION	CONFIDENTIAL

Table of Contents

1. Executive Summary

Why memory is the critical differentiator for production AI agents

AI agents without persistent memory are conversational dead-ends — every interaction starts from zero, users repeat themselves, and agents cannot learn or personalise. AWS re:Invent 2024 and AWS Summit New York 2025 both designated Amazon Bedrock AgentCore Memory as the foundational layer that transforms transactional AI into continuous, evolving relationships. This document is an exhaustive prescriptive reference for implementing memory using the **Strands Agents SDK** on **Amazon Bedrock AgentCore Runtime**, covering every memory flavour, multi-agent patterns, token optimisation, framework trade-offs, EU banking compliance, and production-grade Terraform IaC.

Dimension	Key Finding
Memory Service	AgentCore Memory (GA Oct 2025) — fully managed, abstracted storage with built-in extraction pipelines
Framework	Strands Agents SDK — hook-based architecture is the prescribed integration pattern for AgentCore Memory
Memory Flavours	Short-term (7–365d), Long-term (Summarization, User Preference, Semantic, Episodic), Self-managed
Multi-Agent Patterns	Single-Isolated, Shared Namespace, Publisher/Subscriber, Hub & Spoke, Transaction Ledger
Token Optimisation	Prompt caching (30–70% cost reduction), context compaction, ACON framework, tiered retrieval
EU Banking	Frankfurt (eu-central-1) primary; CMK encryption mandatory; GDPR Art. 17 erasure workflow required
Policy Enforcement	AgentCore Policy (preview Dec 2025) — natural language → Cedar, real-time tool call interception
Key Risk	PII in vector store without redaction — mitigate with mandatory PIIRedactionHook before any memory write
Cost Driver	batch_size=1 is the #1 anti-pattern — use batch_size>=10 and session-end consolidation only

■ Industry signal: S&P; Global Market Intelligence, Experian, Epsilon, Thomson Reuters, and Ericsson are all in active production with AgentCore Memory as of Q4 2025.

2. AgentCore Memory — Architecture Overview

2.1 Five Design Principles

Abstracted Storage	Fully managed — no DynamoDB, OpenSearch, or Redis to manage. Single API surface.
Security by Default	Encrypted at rest and in transit. AWS-managed keys default; CMK mandatory for EU banking.
Continuity	Events stored chronologically. Session branching supports parallel task execution within one actor.
Hierarchical Namespaces	actor_id → namespace → memory_id hierarchy; IAM conditions on namespaces for RBAC.
Scalable Retrieval	Internally managed vector embeddings for semantic search — no embedding model to operate.

2.2 Core Components

Component	Type	Lifecycle	Storage
Memory Resource	Container	Persistent	Logical container — defines retention and encryption
Events	Short-term	7–365 days (configurable)	Encrypted managed store; append-only
Memories	Long-term	Until explicit delete	Vector-indexed for semantic search
Consolidation Job	Process	Post-session or scheduled	Async extraction — triggers from EventBridge
Retrieval API	Interface	Request-scoped	Reads from vector index; returns ranked results
Namespace	Access Ctrl	Persistent	IAM-enforced — actor_id / namespace / memory_id

2.3 Memory Resource Lifecycle

- Agent session starts — actor_id authenticated via Cognito / OIDC (never user-provided)
- MemoryRetrievalHook fires — semantic search over long-term memories for current query
- Retrieved memories injected into system prompt (RAG pattern before model call)
- User message arrives — PIIRedactionHook fires before event is written to memory
- ConsentCheckHook validates GDPR lawful basis exists for this actor_id
- Events buffered (batch_size) and flushed to short-term store as a single API call
- Agent generates response — tool calls intercepted in real time by AgentCore Policy
- AfterInvocationHook fires — interaction persisted; checkpoints saved after tool calls
- Session ends — EventBridge triggers async consolidation job
- Consolidation extracts long-term memories → stored in vector index for future sessions

3. Memory Types — Complete Taxonomy

3.1 Short-Term Memory (Working Memory)

Short-term memory stores raw conversation events chronologically. It is the foundation of all other memory types — long-term memories are derived from it by the consolidation job. The **AgentCoreMemorySessionManager** in the Strands SDK handles buffering, flushing, and session lifecycle automatically.

Attribute	Value	Notes
Retention Range	7 days to 365 days	Set per Memory Resource at creation time
Scope	Single session or Cross-session	Cross-session requires consistent IdP-issued actor_id
Session Branching	Supported	Parallel tasks within same actor — e.g. 3 concurrent drafts
batch_size	1 to N messages	batch_size=10+ strongly recommended — reduces API calls 90%
Encryption	AWS-managed or CMK	CMK mandatory for EU banking and regulated workloads
Session Isolation	Complete per session_id	Different sessions cannot read each other's events

■ **Single vs Cross-Session:** Use single session for one-off transactions. Use cross-session (consistent actor_id) for relationship banking, personal advisors, and any use case where continuity across days or weeks improves quality.

3.2 Long-Term Memory Strategies

Strategy	What It Extracts	Best For	Key Risk	GDPR Basis
SUMMARIZATION	Condensed highlights of session	Long advisory sessions, troubleshooting	May lose nuanced detail	Legitimate interest
USER_PREFERENCE	Likes, dislikes, behavioural patterns	Personalisation, retail and wealth banking	Stale preferences if no TTL configured	Consent required
SEMANTIC	Factual entities, relationships, domain	KYC facts, product knowledge, org hierarchy	Facts may become outdated without refresh	Legitimate interest
EPISODIC	Episodes: context, reasoning, outcomes	Pattern learning, adaptive agents	Risk of encoding biased decisions	Legitimate interest
SELF-MANAGED	Anything — custom Lambda pipeline	Full compliance control, domain-specific	Higher engineering overhead	Operator controls fully

3.3 Episodic Memory (GA December 2025)

A dedicated **reflection agent** analyses structured episodes to extract broader patterns. When similar tasks arise, the main agent retrieves learnings to improve decision-making consistency. AWS demonstrated this at re:Invent: an agent learned from a solo trip booking, then automatically adjusted timing for a family trip three months later.

■ **EU Banking Warning:** Episodic memory influencing credit or fraud decisions must provide a human review pathway (GDPR Art. 22). Reflection agent reasoning must be logged for explainability (MiFID II). Quarterly fairness audits required per EBA ML Guidelines.

3.4 Checkpointing Pattern

AgentCore Runtime supports up to 8-hour execution windows. Write a checkpoint event to AgentCore Memory after each major tool call — using the short-term event store as a durable saga log. Agents can resume after failure without replaying expensive tool calls.

3.5 Retention Period Decision Matrix

Use Case	Memory Type	Retention	GDPR Lawful Basis	Regulatory Driver
Customer support chat	Short-term	7 to 30 days	Legitimate interest	CX best practice
Loan application workflow	Short-term + Checkpoint	90 days	Contract performance	Dispute resolution
Wealth management prefs	Long-term (Preference)	2 years	Consent	MiFID II suitability
Fraud investigation	Long-term (Semantic)	7 years	Legal obligation	5AMLD / GDPR Art. 6(c)
Trading preferences	Long-term (Preference)	1 year	Consent + Legal obligation	MiFID II Art. 25
KYC entities	Long-term (Semantic)	5 years	Legal obligation	4AMLD / 5AMLD
Episodic patterns	Episodic	1 year + review	Legitimate interest	EBA ML Guidelines
Complaint history	Long-term (Semantic)	3 years	Legal obligation	FCA DISP / EU Directive

4. Multi-Agent Memory Patterns — Pros & Cons

4.1 Single Agent — Isolated Namespace

Each agent owns a dedicated Memory Resource. No sharing between agents. actor_id maps to a specific user. Simplest and most secure pattern. Start here for any new project.

ADVANTAGES	DISADVANTAGES
✓ Strongest isolation — no risk of cross-agent data contamination ✓ Simplest IAM policy — one role, one resource, one namespace ✓ Easiest right-to-erasure — delete one Memory Resource or namespace ✓ Lowest operational complexity — no inter-agent coordination needed	✗ Knowledge siloed — Agent B cannot benefit from facts known to Agent A ✗ Duplicate storage — same user facts stored N times across N agents ✗ No collaborative intelligence — agents cannot build on each other's reasoning

■ **Recommendation:** Default starting point. Use for support bots, simple chatbots, single-function agents.

4.2 Shared Memory — Publisher / Subscriber

A dedicated writer agent (e.g. KYC Collector) persists facts. Downstream reader agents (Credit, Risk, Compliance) are read-only consumers. Hierarchical namespaces + IAM conditions enforce access control.

ADVANTAGES	DISADVANTAGES
✓ Single source of truth — KYC facts written once, consumed by many agents ✓ Read agents fully decoupled from write logic — independent scalability ✓ Fine-grained IAM: reader role cannot accidentally overwrite writer namespace ✓ Namespace-level audit trail for compliance — clear data provenance	✗ Writer agent is a single point of failure for downstream data quality ✗ Reader agents may act on stale data if consolidation job is delayed ✗ Namespace design requires upfront information architecture investment ✗ Cross-agent trust model must be established and maintained in IAM

■ **Recommendation:** Ideal for KYC-driven workflows. Writer = KYC/onboarding. Readers = all downstream decision agents.

4.3 Multi-Write Hub & Spoke

Sub-agents write to their own namespaces in parallel. An orchestrator consolidates all outputs into a final shared namespace after all tasks complete. Eliminates race conditions from concurrent writes to the same namespace.

ADVANTAGES	DISADVANTAGES
✓ Concurrency-safe — no race conditions or last-write-wins conflicts ✓ Modular — each sub-agent owns its output namespace completely ✓ Orchestrator provides quality gate before promoting to shared namespace ✓ Supports complex parallel workflows (fraud + credit + compliance simultaneously)	✗ Higher orchestration complexity — must track all sub-agent completion status ✗ Consolidation latency — shared namespace not updated until all agents finish ✗ More IAM roles to manage — one writer role per sub-agent type minimum ✗ Partial failure risk if orchestrator crashes mid-consolidation (saga pattern needed)

■ **Recommendation:** Use for complex loan origination, KYC review, or compliance workflows with 3+ specialist sub-agents.

4.4 Transaction Ledger Pattern

All memory events treated as an append-only, immutable ledger. Every decision, tool call, and extraction is timestamped and cryptographically signed. Required for MiFID II, DORA, and AML audit compliance.

ADVANTAGES	DISADVANTAGES
✓ Full auditability — every decision traceable to specific memory inputs ✓ Tamper-evident — HMAC signature on each event detects modification ✓ Regulatory ready — satisfies MiFID II record-keeping and AML audit requirements ✓ Replay capability — reconstruct agent reasoning for any past session	✗ Storage cost grows unbounded — must tier to S3 WORM for long-term retention ✗ Query complexity — replaying ledger requires parsing ordered event stream ✗ HMAC signature computation adds latency per event write ✗ No correction capability — errors must be compensated, not overwritten

■ **Recommendation:** Mandatory for EU banking agents making or influencing financial decisions. Pair with S3 Object Lock WORM.

5. Memory Processors & Extractors

The correct processing order is non-negotiable: **Ingest Events** → **PII Redact** → **Entity Extract** → **Summarize** → **Consolidate** → **Vector Store** → **Retrieve**. Skipping or reordering any step creates GDPR exposure.

5.1 PII Redaction Pipeline

Method	Detection Capability	Latency	EU Banking Fit
Regex patterns	IBAN, card numbers, NIN, passport — format-specific	~1ms	Necessary but insufficient — misses contextual PII
Amazon Comprehend	Names, addresses, emails, phones, dates, organisations	50-200ms	Good foundation — AWS-native; easy Strands hook integration
Presidio (OSS)	Contextual NER + regex — multilingual EU support	100-500ms	Excellent — handles German, French, Spanish PII natively
Custom Lambda	Domain-specific: ISIN, LEI, BIC/SWIFT, product codes	Varies	Required for financial services — add after Comprehend base
Bedrock Guardrails	Output-side filtering — catches PII in agent responses	10-50ms	Complementary only — not a replacement for write-time redaction

5.2 Entity Extractor

After PII redaction, the entity extractor identifies structured, non-personal facts for semantic memory storage. Banking-relevant types: financial product interests, risk appetite, life events, stated goals, complaint topics, relationship signals. Use a self-managed strategy Lambda with a structured JSON extraction prompt targeted to your domain.

5.3 Summarizer

Approach	Trigger	Token Reduction	Quality Impact	Best For
AgentCore SUMMARIZATION strategy	Post-session (managed)	30-60%	High — managed by service	Standard advisory sessions
Strands context compaction	70% context window threshold	40-65%	Medium-High	Long running sessions (1h+)
Rolling window (last N turns)	Every M turns	Up to 80%	Low — loses early context	High-frequency bots, cost-sensitive
ACON optimised compaction	Post-failure analysis	26-54%	Very High — failure-driven	Complex multi-step agent pipelines

5.4 Consolidation Job

Trigger consolidation asynchronously on session-end via EventBridge. Never trigger per-message. The job must: verify PII redaction, deduplicate memories, resolve conflicts (latest timestamp wins), run extraction strategy, write to vector index.

■ **Anti-pattern:** Per-message consolidation multiplies cost significantly. Always use EventBridge rule on the 'AgentCore Session Ended' event.

6. Framework Comparison — Pros & Cons

6.1 Comparative Analysis

Framework	Architecture	Key Advantages	Key Disadvantages	Best For
AgentCore Memory (AWS)	Fully managed, serverless. Vector store + event log behind single API.	• Zero infrastructure to operate • Native AWS IAM and KMS • VPC / PrivateLink support • Strands SDK native hooks • GDPR-ready EU regions • Episodic + Semantic built-in	• AWS lock-in • Limited graph memory • Cross-region inference risk for EU • Newer service — GA Oct 2025 • Less customisation vs OSS	AWS teams; EU banking; regulated industries; zero infra overhead
Mem0 (OSS / Managed)	Hybrid: vector DB + LLM extraction. Graph memory in Pro tier.	• 48K+ GitHub stars — largest community • Framework-agnostic • Self-hosted or managed options • Strong personalisation • LangChain / LlamaIndex native	• Graph memory costs extra (Pro) • 7-8s retrieval latency at scale • No native Strands hook integration • GDPR burden on operator • Separate infra for self-hosted	Non-AWS stacks; prototyping; personalisation-heavy agents
Zep / Graphiti (Graph)	Knowledge graph with ontology-aware edges. Temporal reasoning.	• Sub-100ms retrieval (Zep Cloud) • Best temporal reasoning • SOC2, HIPAA, GDPR certified • Strong for audit trails	• Manual graph node/edge management • $15/M tokens — premium • Smaller ecosystem than Mem0 • Steeper learning curve	Compliance agents; policy-tracking; temporal fact evolution

Framework	Architecture	Key Advantages	Key Disadvantages	Best For
LangMem (LangChain)	Library, not a service. LangGraph native. Local or PostgreSQL storage.	• Free and open source (MIT) • Zero vendor lock-in • Background memory manager • LangGraph create_react_agent native	• LangChain ecosystem only • No hosted option • Python only — no TypeScript • Structural limits at scale	LangGraph teams; budget-constrained; full data ownership
Letta / MemGPT	Tiered memory. Agents actively manage own memory blocks.	• Unlimited context — agents self-manage • Agents decide what to retain • Self-hosted free tier • Strong for long-horizon tasks	• Framework lock-in — Letta runtime required • File traversal — slow at scale • No LangChain / CrewAI native support • Niche ecosystem	Research agents; long-horizon tasks; Letta-committed teams

6.2 Decision Matrix

Requirement	AgentCore	Mem0	Zep	LangMem	Letta
AWS-native integration	Best	Manual	Manual	Manual	None
EU data residency (GDPR)	eu-central-1	Self-hosted	SOC2/GDPR	Full control	Full control
Zero infrastructure	Fully managed	Managed tier	Managed tier	Self-managed	Self-managed
Strands framework support	Native SDK	Manual hooks	Manual hooks	N/A	N/A
Retrieval latency (p99)	~200ms	7-8 seconds	<100ms (Cloud)	50-200ms	Varies
Graph / temporal reasoning	Limited	Pro tier	Best	None	Basic
Framework flexibility	Any framework	Any	Any	LangGraph only	Letta only
Production maturity (2026)	GA Oct 2025	4+ years	Production	Maturing	Maturing

■ **Recommendation for AWS / EU Banking:** AgentCore Memory with Strands SDK is the correct default. Tightest AWS security posture, native Strands integration, EU region compliance, and zero infra overhead. Complement with Zep only if temporal graph reasoning is required.

7. Memory & Token Optimisation Strategies

Advanced techniques for reducing cost and latency without sacrificing memory quality

Token cost is the primary variable operating expense for memory-enabled agents. Research shows agentic workloads spend 30-70% of time in I/O wait, and **65% of enterprise AI failures in 2025 were caused by context drift or memory loss** during multi-step reasoning — not raw context exhaustion (Zylos Research, Feb 2026).

7.1 Context Drift & The 70% Rule

Context drift occurs when model reasoning diverges from the original task as conversation history grows. Research (Chroma 2025) demonstrates performance degradation accelerates beyond 30,000 tokens even in models with much larger context windows. The recommended compaction trigger is **70% of available context budget** — before drift begins, not after.

7.2 Prompt Caching — 30–70% Cost Reduction

Prompt caching reuses previously computed KV tensors from attention layers, eliminating redundant computation on repeated prompt prefixes. Research across 500+ agent sessions shows statistically significant cost reductions across all major providers. For Strands + Bedrock, use **BedrockModel CacheConfig(strategy='auto')**.

Caching Strategy	What Gets Cached	Cost Reduction	Trade-off
System prompt caching	Static system prompt + rules	20-40% input tokens	Cache invalidated if prompt changes — keep it stable
Retrieved memory caching	Previously fetched long-term memories (15 min)	10-25% retrieval calls	Stale memories possible if prefs updated mid-session
Tool schema caching	Tool definitions sent on every request	5-15% input tokens	Minimal risk — schemas rarely change in production
Full prefix caching (auto)	Longest stable prefix auto-detected	30-70% input tokens	Breaks if dynamic content is inserted early in prompt

7.3 Context Compaction — ACON Framework

ACON (Agent Context Optimization, arXiv Oct 2025) treats compression as an optimisation problem. It identifies where full context succeeded but compressed context failed, uses an LLM to find what was lost, then updates compression prompts to preserve that information. Results show **26-54% reduction in peak token usage** without quality degradation.

Method	How It Works	Reduction	Quality Impact
Truncation	Drop oldest messages until under limit	Up to 90%	High loss — not recommended for production
Rolling summary	Replace oldest N turns with LLM summary	30-60%	Low-medium loss — acceptable for simple bots
Anchored iterative summarization	Preserve task anchors + compress middle context	40-65%	Low loss — good for most advisory agents
ACON optimised compaction	Failure-driven guideline update + distillation	26-54%	Minimal loss — best quality-to-cost ratio
Anthropic compact-2026-01-12	Provider-native automatic compaction API	Varies	Managed by provider — simplest to adopt

7.4 Retrieval Caching & Batch Write Optimisation

Retrieval caching: Implement a 15-minute cache keyed by actor_id. Invalidate on any memory write event. Eliminates 70-90% of vector store queries for high-frequency agents.

Batch write optimisation: batch_size=1 generates one API call per message — the most expensive pattern. batch_size=10 reduces requests by 90% with no quality trade-off. Always use a context manager (with block) or explicit close() to flush buffer.

■ **Cost example:** 100 messages at batch_size=1 = 100 API calls. batch_size=10 = 10 API calls — identical data, 90% fewer requests. At 10,000 concurrent users, this difference is substantial.

7.5 Tiered Memory Retrieval — Intelligent Context Construction

Injecting all retrieved memories into every prompt adds noise and increases cost. A tiered retrieval strategy targets approximately 900 tokens — high relevance, low noise.

Tier	Memory Type	Token Budget	Retrieval Strategy
1 — Always inject	User identity facts (account type, tier)	~50 tokens	Deterministic — no search required
2 — Session context	Current session summary (cross-session)	~200 tokens	Latest session summary only
3 — Preferences	Semantically relevant to current query	~300 tokens	Top-3 semantic search results
4 — Domain facts	Entities relevant to current query	~200 tokens	Top-5 semantic search results
5 — Episodic hints	Similar past task patterns (if enabled)	~150 tokens	Top-2 episodic matches
Total budget	All tiers combined	~900 tokens	High relevance, within all context limits

8. Strands Framework Best Practices

Mandatory hooks, sub-agent skills, system prompt rules, and repository structure

Strands Agents uses a hook-based architecture — callback functions executed at specific points in the agent lifecycle. This cleanly separates memory management from business logic, making code testable and auditable. The four hooks below are mandatory.

8.1 Mandatory Hooks

PIIRedactionHook · *MessageAddedEvent (fires before write to memory)*

Intercepts every message before it reaches AgentCore Memory. Applies Comprehend + Presidio redaction pipeline. GDPR Art. 25 (Privacy by Design) requires this as the first operation on any user-generated content.

```
class PIIRedactionHook: def __init__(self, redactor: PIIRedactor): self.redactor = redactor def
on_message_added(self, event: MessageAddedEvent): if event.message.role in ('user', 'assistant'):
event.message.content = self.redactor.redact( event.message.content ) return event # modified event
continues to memory write
```

MemoryRetrievalHook · *MessageAddedEvent (on user message — fires before model call)*

Performs semantic search over long-term memories and injects ranked results into the system prompt before the model sees the user message. This is the RAG pattern applied to personalised agent memory.

```
class MemoryRetrievalHook: def __init__(self, memory_client, memory_id, actor_id): self.memory =
memory_client self.memory_id = memory_id self.actor_id = actor_id # MUST come from IdP — never user input
def on_message_added(self, event: MessageAddedEvent): if event.message.role == 'user': memories =
self.memory.retrieve_memories( memory_id=self.memory_id, namespace=f'users/{self.actor_id}',
search_query=event.message.content, max_results=5 ) event.agent.system_prompt = (
self._format_memories(memories) + '\n\n' + event.agent.system_prompt )
```

MemoryPersistenceHook · *AfterInvocationEvent (fires after agent response)*

Persists each interaction to AgentCore Memory after the agent response. Handles checkpointing after tool-heavy turns and notifies EventBridge for session-end consolidation.

```
class MemoryPersistenceHook: def __init__(self, session_manager, checkpoint_client=None): self.session =
session_manager self.checkpoint = checkpoint_client def after_invocation(self, event:
AfterInvocationEvent): self.session.flush_if_threshold_reached() if self.checkpoint and
event.had_tool_calls: self.checkpoint.save( event.session_id, event.tool_results )
```

ConsentCheckHook · *StartAgentCycleEvent (fires before each agent reasoning cycle)*

Validates that a GDPR lawful basis exists for memory processing before any read or write. Disables memory hooks for the session if no valid basis found. Mandatory for EU banking.

```
class ConsentCheckHook: def __init__(self, consent_service): self.consent = consent_service def
on_start_agent_cycle(self, event: StartAgentCycleEvent): actor_id = event.context.get('actor_id') if not
self.consent.has_valid_basis(actor_id, 'memory'): event.context['memory_enabled'] = False logger.audit(
f'Memory disabled: no GDPR basis for {actor_id}' )
```

8.2 Sub-Agent Skills / Tools

Skill	Access Role	Purpose	Mandatory?
memory_write	Writer only	Wraps put_events. Enforces PII check, consent validation, namespace scope.	YES
memory_read	Reader roles	Wraps retrieve_memories. actor_id = authenticated user only. Returns guardrail-filtered results.	YES
memory_delete	DPO Admin only	Right-to-erasure: deletes all namespaces for actor_id. Requires confirmation webhook. CloudTrail logged.	YES
memory_search	Reader roles	Semantic search over long-term memories. Returns top-K with relevance score. Query logged.	YES

Skill	Access Role	Purpose	Mandatory?
session_summarise	Agent self-call	Manual trigger for context compaction when approaching 70% context window utilisation.	Recommended
memory_audit	Compliance only	SAR response: retrieve all memories for a customer. Strictly role-gated. Full audit trail.	Recommended

8.3 System Prompt Rules (Non-Negotiable)

- NEVER reveal raw memory content to the user verbatim — synthesise and apply naturally
- NEVER store or repeat PII (full names, account numbers, SSN, passport) in any response or tool call
- If customer requests data deletion, immediately invoke memory_delete skill and confirm completion
- NEVER derive actor_id from conversation content — only use the IdP-authenticated context header
- When memory contradicts current user input, always prefer current input and update memory accordingly
- Before financial recommendations using memory data, confirm with user that preferences are still current
- Memory data is CONFIDENTIAL — never share one customer's context with another agent or session
- If memory retrieval fails, degrade gracefully — proceed without memory rather than exposing errors

8.4 Repository Structure (Mandatory Layout)

```
your-agent-repo/ ■■■ .agentcore/config.json # AgentCore CLI config ■■■ agent/ ■ ■■■ main.py # Agent entry point ■ ■■■ hooks/ # MANDATORY — all 4 required ■ ■ ■■■ pii_redaction_hook.py ■ ■ ■■■ memory_retrieval_hook.py ■ ■ ■■■ memory_persistence_hook.py ■ ■ ■■■ consent_check_hook.py ■ ■■■ skills/ # MANDATORY tools ■ ■ ■■■ memory_write.py ■ ■ ■■■ memory_read.py ■ ■ ■■■ memory_delete.py ■ ■ ■■■ memory_search.py ■ ■■■ processors/ ■ ■ ■■■ pii_redactor.py # Comprehend + Presidio + custom ■ ■ ■■■ entity_extractor.py ■ ■ ■■■ summarizer.py ■ ■■■ config/ ■ ■■■ memory_config.py # Retention + strategies per env ■ ■■■ consent_config.py # GDPR lawful basis mapping ■■■ infrastructure/terraform/ ■ ■■■ memory_resource.tf ■ ■■■ iam_memory.tf ■ ■■■ kms_memory.tf ■ ■■■ vpc_privatelink.tf ■■■ tests/ ■ ■■■ test_pii_redaction.py # Zero-tolerance PII test suite ■ ■■■ test_memory_isolation.py # Cross-tenant isolation tests ■ ■■■ test_right_to_erasure.py ■■■ docs/ ■■■ GDPR_DPIA.md # Required — DPO sign-off ■■■ RUNBOOK_SAR.md # SAR handling runbook
```

9. EU Banking, GDPR & Regulatory Compliance

Mandatory controls for operating AgentCore Memory in European financial services

■ EU-based banks must simultaneously satisfy GDPR (2018), EU AI Act (2024), DORA (2025), EBA Guidelines on ML/AI, MiFID II, 5AMLD, and national supervisory requirements.

9.1 GDPR Obligations Mapped to AgentCore

GDPR Article	Obligation	AgentCore Implementation
Art. 5 — Data Minimisation	Only store memory relevant to the lawful purpose. No blanket retention.	Configure minimum strategies. Set aggressive TTLs. Quarterly audit.
Art. 6 — Lawful Basis	Each memory type needs a documented legal basis before processing.	ConsentCheckHook validates basis per actor_id. Document in consent_config.py.
Art. 17 — Right to Erasure	Customer can demand deletion of all their data within 30 days.	memory_delete skill + SAR runbook. Cascade delete events, memories, vector entries.
Art. 22 — Automated Decisions	Decisions based solely on automated processing require human review.	Episodic memory influencing credit or fraud must log reasoning + provide review path.
Art. 25 — Privacy by Design	Privacy protections embedded from the start — not added as an audit.	PIIRedactionHook fires BEFORE any memory write. Architectural position — mandatory.
Art. 32 — Security	Appropriate measures to protect personal data.	CMK encryption + VPC + PrivateLink + MFA + CloudTrail. All mandatory for banking.
Art. 35 — DPIA	Impact assessment required before deploying high-risk AI.	Complete DPIA template in docs/GDPR_DPIA.md. DPO sign-off gate before production.

9.2 DORA, EBA ML Guidelines & MiFID II

Regulation	Requirement	Memory-Specific Control
DORA Art. 6 — ICT Risk	Document and manage ICT risk including AI systems.	VPC + PrivateLink. Memory access in CloudTrail. ICT risk register entry required.
DORA Art. 11 — Backup	Business continuity for critical ICT assets.	Cross-region backup plan. S3 cross-region replication for event archive.
EBA ML §4.3	Model explainability for AI-driven decisions.	AgentCore Observability logs WHY a memory influenced a decision. Trace IDs required.
EBA ML §5.1	Bias and fairness monitoring for ML models.	Quarterly fairness audit on episodic memory patterns across demographic segments.
MiFID II Art. 25	Suitability records retained 5 years minimum.	Suitability memory in Semantic strategy. Immutable. 5-year TTL minimum.
5AMLD — AML	KYC data retained 5 years post-relationship end.	KYC Semantic memories in compliance Memory Resource. 7-year retention. WORM.

9.3 EU Region Constraints

■ **Critical:** Disable cross-region inference in Memory Resource config or implement an AWS Organization SCP to block cross-region AgentCore API calls. Primary: eu-central-1 (Frankfurt). Secondary: eu-west-1 (Ireland).

9.4 Required Controls Checklist

Technical Controls

- KMS CMK with EU key material only
- VPC + PrivateLink for all AgentCore Memory API calls
- PIIRedactionHook tested — zero PII in vector store validated
- IAM roles: separate writer / reader / DPO-admin — no wildcards
- CloudTrail enabled — all memory API calls logged, 7-year S3
- Amazon Macie weekly scan on any S3 event archive
- Right-to-erasure tested end-to-end (SAR drill completed)
- Session isolation test: actor_id collision returns empty results
- Cross-region inference disabled in Memory Resource config
- KMS key rotation enabled (90-day auto-rotate)

Process Controls

- DPIA completed and signed by DPO
- Data Processing Agreement (DPA) with AWS executed
- Memory retention schedule documented and approved
- Consent management integration tested (if applicable)
- SAR runbook written and rehearsed with DPO team
- Quarterly bias audit process established for episodic memory
- Annual penetration test scope includes memory attack surface
- ICT risk register updated with AgentCore Memory entry (DORA)
- AgentCore Evaluations dashboard live with memory metrics
- Incident response runbook for memory data breach

10. Security, Policy & Threat Model

10.1 AgentCore Policy (Cedar Language)

AgentCore Policy (preview Dec 2025) intercepts every tool call in real time. Natural language policies auto-convert to Cedar without writing formal policy code. Enforcement operates in milliseconds.

- "Block writing to memory namespaces outside the agent's assigned scope"
- "Prevent memory reads where the requesting agent's role does not match the namespace ACL"
- "Require PII redaction confirmation metadata before any memory write event"
- "Block episodic memory reads if customer has invoked their GDPR right to be forgotten"
- "Prevent credit agent from reading fraud investigation memories unless escalation flag is set"
- "Limit preference memory writes to max 50 entries per customer to prevent data bloat"
- "Alert compliance team if AML entity extracted to memory matches sanctions watchlist"

10.2 Memory Attack Surface — AgentLeak Taxonomy (2026)

Attack Family	Memory Vector	Description	Mitigation
F3: Memory Injection	Event write surface	Attacker crafts user input that survives PII redaction, planting false memories in vector store.	Input validation Lambda + adversarial content detector. Test quarterly.
F3: Persistence Hijack	Consolidation	Malicious event content extracted as 'preference' and replayed in future sessions as trusted context.	Schema validation on extracted memories. Allowlist valid entity types.
F3: Cross-session Leak	actor_id derivation	actor_id collision enables reading another customer's memories when actor_id comes from user input.	actor_id = Cognito sub ONLY. Enforce via ConsentCheckHook and IAM condition.
F1: Prompt Override	Retrieval injection	User prompt manipulates agent into outputting raw memory contents verbatim.	System prompt rule: never output raw memories. Bedrock Guardrails output filter.
F2: Tool Surface	Pre-write pipeline	Tool output poisoned with false data before memory write, bypassing user-input validation.	Tool output sanitisation hook. Validate against expected schema.

10.3 Encryption Architecture

Layer	Encryption Key	Key Management	EU Banking Requirement
Events (short-term)	Customer-managed KMS (CMK)	Rotate 90 days; eu-central-1 key only	MANDATORY
Memories (long-term)	Same CMK + encryption context	Context = {customer_id, namespace}	MANDATORY
Vector Embeddings	AWS-managed or CMK	CMK preferred for banking grade	Strongly Recommended
In-Transit	TLS 1.3 minimum	Enforced by PrivateLink — no public net	MANDATORY
CloudWatch Logs	CMK log group encryption	Separate key from data encryption key	MANDATORY
S3 WORM Archive	SSE-KMS + Object Lock	COMPLIANCE mode — 7-year retention lock	MANDATORY (AML)

11. Cost Analysis & Optimisation

Component	Pricing Dimension	Primary Cost Driver	Optimisation Strategy
Memory Resource	Per resource / month	Number of logical containers	Consolidate to fewer resources per product line
Event Storage	Per GB stored	Raw conversation volume x retention	Set aggressive short-term TTL (30d vs 365d default)
Memory Storage	Per GB stored	Extracted memory volume x strategies	TTL on stale preferences; summarise rather than accumulate
Consolidation	Per extraction run	Frequency x active daily sessions	Post-session trigger only — never per-message
Retrieval API	Per 1K requests	Memory lookups per agent turn	Cache retrieved context 15 min; tiered retrieval budget
KMS CMK calls	Per API call	Encrypt/decrypt per event write	Envelope encryption — one data key per session
Runtime	Active CPU/memory	Agent compute time + input tokens	batch_size>=10; prompt caching cuts input tokens 30-70%

COST ANTI-PATTERNS

- batch_size=1 — multiplies API costs 5-15x
- 365-day retention on all memory types
- Real-time consolidation after every message
- No retrieval cache — fetching preferences every turn
- Individual KMS call per event (not envelope encryption)
- All 4 strategies on low-volume simple chat bots
- No context compaction on 200-turn sessions

COST BEST PRACTICES

- batch_size=10+ — 90% fewer API requests
- Tiered retention: 30d short-term, 1y long-term
- Session-end consolidation via EventBridge rule
- 15-minute retrieval cache keyed on actor_id
- Envelope encryption: one KMS call per session
- Strategy selection per use case — Summarization for simple bots
- CacheConfig(strategy='auto') — 30-70% token reduction

12. Project Journey — PoC to Production

PHASE 1 **Memory-Enabled Proof of Concept** Weeks 1-4 MILESTONE Agent maintains context across turns within a single session	• Create AgentCore project (agentcore create) in eu-central-1 • Provision Memory Resource with SUMMARIZATION strategy only • Implement all 4 mandatory hooks (Consent stubbed for PoC) • Single-session memory with 30-day retention configured • Use synthetic data only — zero real PII in PoC environment • Deploy to AgentCore Runtime; validate retrieval latency <200ms p99 • Write hook unit tests with mocked memory client
PHASE 2 **Cross-Session + Long-Term Memory** Weeks 5-8 MILESTONE Agent recalls user preferences across separate sessions	• Add USER_PREFERENCE strategy to Memory Resource • Wire actor_id = authenticated Cognito sub (never user-provided) • Implement PII redaction Lambda (Comprehend + domain patterns) • Enable cross-session memory: consistent actor_id across sessions • Build right-to-erasure: memory_delete skill + namespace cascade • Wire ConsentCheckHook to real consent management service • GDPR consent validation integration test + DPO retention review
PHASE 3 **Multi-Agent Memory Architecture** Weeks 9-12 MILESTONE Multi-agent workflow with governed shared memory and policy enforcement	• Design namespace hierarchy for multi-agent access (Hub and Spoke) • Implement IAM roles: one writer, multiple readers, DPO-admin • Add self-managed strategy with entity extractor Lambda • Orchestrator agent consolidates sub-agent namespace outputs • Enable AgentCore Policy: namespace access control Cedar rules • CloudTrail audit logging for all memory API calls configured • Performance test: 100 concurrent agents on shared Memory Resource • Add tiered retrieval logic — context budget approximately 900 tokens
PHASE 4 **Episodic Memory + Production Hardening** Weeks 13-16 MILESTONE EU banking grade production — DPO approved for go-live	• Enable episodic memory on high-value workflow agents only • DPIA review with DPO — sign off on all memory types • CMK encryption with 90-day key rotation configured • VPC + PrivateLink for all AgentCore endpoints • S3 WORM archive for AML-retention event export • AgentCore Evaluations dashboard live — all 9 memory metrics active • Full penetration test (memory injection, cross-session leak surface) • SAR runbook rehearsal — right-to-erasure end-to-end drill with DPO

13. Evaluation Framework

AgentCore Evaluations (preview Dec 2025) + memory-specific quality metrics

Metric	Definition	Target	Alert Threshold
Memory Retrieval Relevance	Cosine similarity between retrieved memory and current query	above 0.85	below 0.75 — tune retrieval strategy
PII Leakage Rate	Percentage of memory reads containing unredacted PII	0.00%	Above 0% — CRITICAL, page DPO immediately
Cross-Session Recall	Percentage of correctly recalled preferences from prior session	above 90%	below 80% — review extraction strategy
Memory Staleness Rate	Percentage of retrieved memories older than defined TTL	below 5%	above 10% — run TTL purge job
Namespace Isolation	Percentage of out-of-scope access attempts blocked	100%	below 100% — CRITICAL, audit IAM policies
Preference Extraction Precision	Percentage of extracted prefs matching what user stated	above 85%	below 75% — tune extractor prompt
Erasure Completeness	After deletion: 0 memories retrievable for actor_id	100%	below 100% — CRITICAL, potential GDPR breach
Episodic Bias Score	Fairness delta across demographic segments in decisions	Delta <5%	Delta >10% — suspend episodic, audit immediately
Retrieval Latency p99	End-to-end: retrieve + inject + model first token time	below 500ms	above 1s — check vector index and batch config

■ **Continuous Evaluation Loop:** Deploy > CloudWatch Dashboard > AgentCore Evaluations (10% daily sample) > Metric Breach Alert > Engineering + DPO Triage > Remediate > Re-evaluate > Close loop. PII leakage and namespace isolation breaches are P0 incidents requiring DPO notification under GDPR Art. 33 within 72 hours.

14. Terraform IaC Reference

Production-grade IaC for EU banking grade AgentCore Memory

Memory Resource (memory_resource.tf)

```
resource "aws_bedrockagentcore_memory" "banking_memory" { name = "${var.project}-${var.env}-memory" description = "EU Banking agent memory — GDPR compliant" event_expiry_duration = var.event_retention_days # 30 default, 90 for loans encryption_key_arn = aws_kms_key.memory_cmk.arn memory_strategies { summarization_memory_strategy { name = "ConversationSummary" } } # Add USER_PREFERENCE and SEMANTIC only for relationship banking agents tags = { DataClass="RESTRICTED", GDPRScope="true", RetentionOwner="DPO" } }
```

KMS CMK (kms_memory.tf)

```
resource "aws_kms_key" "memory_cmk" { description = "AgentCore Memory CMK — EU Banking" deletion_window_in_days = 30 enable_key_rotation = true # Auto-rotate every 90 days multi_region = false # Single EU region key (GDPR) # Key policy: deny requests from outside eu-central-1 / eu-west-1 tags = { GDPRScope = "true", DataClass = "RESTRICTED" } }
```

IAM Roles (iam_memory.tf)

```
# Writer agent — writes to "users/*" namespace only resource "aws_iam_policy" "memory_writer" { policy = jsonencode({ Statement = [{ Effect = "Allow" Action = ["bedrock-agentcore:PutMemoryEvents"] Resource = aws_bedrockagentcore_memory.banking_memory.arn Condition = { StringLike = { "bedrock-agentcore:namespace" = "users/*" }} }] }) } # DPO admin — right-to-erasure only; EU regions only resource "aws_iam_policy" "memory_dpo_admin" { policy = jsonencode({ Statement = [{ Effect = "Allow" Action = ["bedrock-agentcore:DeleteMemory", "bedrock-agentcore:DeleteMemoryNamespace"] Resource = "*" Condition = { StringEquals = { "aws:RequestedRegion" = ["eu-central-1","eu-west-1"] }} }] }) }
```

VPC + PrivateLink (vpc_privatelink.tf)

```
resource "aws_vpc_endpoint" "agentcore_memory" { vpc_id = aws_vpc.main.id service_name = "com.amazonaws.eu-central-1.bedrock-agentcore-memory" vpc_endpoint_type = "Interface" subnet_ids = aws_subnet.private[*].id security_group_ids = [aws_security_group.agentcore_endpoint.id] private_dns_enabled = true # Endpoint policy restricts to specific IAM writer/reader roles only }
```

15. Risks, Recommendations & Decision Guide

15.1 Risk Register

Risk	Category	Severity	Mitigation	Owner
PII leakage to vector store	GDPR / Security	CRITICAL	PIIRedactionHook mandatory before every write; Macie scan	Engineering
Cross-tenant actor_id collision	Security	CRITICAL	IdP-only actor_id; IAM namespace condition enforced	Security
Missing right-to-erasure	GDPR	CRITICAL	memory_delete skill + SAR runbook + quarterly drill	DPO
Cross-EU region data flow	GDPR	HIGH	Disable cross-region inference; SCP at org level	Cloud Arch
Memory injection attack	Security	HIGH	Input validation hook; adversarial detector; quarterly test	Security
Episodic bias drift	Regulatory / EBA	HIGH	Quarterly fairness audit on episodic patterns	MLOps
Art. 22 automated decision	GDPR / Regulatory	HIGH	Human review path for any automated financial decision	Product / Legal
Context drift beyond 30K tokens	Quality	MEDIUM	70% compaction trigger; Strands built-in compaction	Engineering
Cost overrun from batch_size=1	Operational	MEDIUM	batch_size>=10 enforced; session-end consolidation only	Engineering
Stale preferences without TTL	Quality	MEDIUM	TTL on USER_PREFERENCE; annual user confirmation flow	Product

15.2 When to Use Which Memory Pattern

Use Case	Pattern	Strategies	Avoid
Simple FAQ / single-function	Single agent, 1 session	SUMMARIZATION only	USER_PREFERENCE — unnecessary overhead
Long advisory session (1h+)	Single, cross-session	SUMMARIZATION + compaction	Episodic — complexity without clear benefit
Retail banking personalisation	Single, cross-session	PREFERENCE + SUMMARIZATION	Self-managed — over-engineered for this case
KYC / onboarding workflow	Publisher / Subscriber	SEMANTIC (entity facts)	EPISODIC — compliance risk without fairness audit
Wealth management / relationship	Single or shared	SUMMARIZATION + PREFERENCE + SEMANTIC	Single session — loses cross-visit continuity
Multi-agent loan origination	Hub and Spoke	Self-managed per sub-agent	Direct concurrent writes to shared namespace
AML / compliance investigation	Transaction Ledger	SEMANTIC + S3 WORM archive	EPISODIC — fairness audit required first
Learning / adaptive (R&D;)	Single + Episodic	EPISODIC + SUMMARIZATION	Production without fairness audit plan in place

15.3 Final Recommendations

• **Start simple, add complexity intentionally.** Begin with SUMMARIZATION only. Add USER_PREFERENCE when personalisation value is validated. Enable EPISODIC only after a fairness audit plan is in place.

• **PIIRedactionHook is non-negotiable.** The single highest-impact control. Implement it before any real user data flows through the system.

• **actor_id from IdP only, always.** This is the root cause of cross-tenant memory leaks in every reported production incident. Never derive from conversation content.

• **batch_size=10 is a free performance improvement.** No quality trade-off, no architectural change. Set this in every deployment from day one.

• **Invest in right-to-erasure before launch.** A GDPR Art. 17 request without a tested workflow is a 72-hour incident, not a sprint ticket.

• **Prompt caching is the highest-ROI token optimisation.** CacheConfig(strategy='auto') is a one-line change delivering 30-70% cost reduction with zero accuracy trade-off.

• **AgentCore Memory for AWS-committed EU banks is the correct default.** Native Strands integration, EU regions, CMK, VPC/PrivateLink, and IAM namespace RBAC make it the most defensible architecture.

Document prepared for enterprise AI architecture teams building on AWS Bedrock AgentCore. Recommendations align with AWS Well-Architected Framework, EU AI Act (2024), GDPR (2018), DORA (2025), and EBA Guidelines on machine learning models. Review annually or upon major regulatory update.